

El Roadmap Completo del Analista de Datos y Data Science

Un handbook premium, de nivel empresarial, desde principiante/intermedio hasta Data Analyst profesional — con proyección hacia Data Science.

Compilado el 2026-07-31

Tabla de Contenidos

- 01 · Introducción
- 02 · Cómo Trabajan las Empresas
- 03 · Bases de Datos
- 04 · SQL — El Curso Completo
- 05 · Python
- 06 · Power BI
- 07 · Proyectos Profesionales
- 08 · Machine Learning
- 09 · Portfolio y Carrera

01 • Introducción

Parte 1 de El Roadmap Completo del Analista de Datos y Data Science · Siguiente: [02. Cómo Trabajan las Empresas](#)

 **Qué vas a poder hacer después de este capítulo** Explicar, en una entrevista o a alguien no técnico, exactamente qué hace diferente un Data Analyst de un Data Scientist, un Data Engineer y un Analytics Engineer — y elegir cuál de esos caminos de carrera perseguir, con una idea realista de habilidades, salario y trabajo diario.

1.1 ¿Qué es el dato?

Olvidate de la definición de manual por un segundo. Adentro de una empresa, **el dato es un escape**. Cada acción que hace un usuario, cada sistema que corre, cada transacción que se cierra — deja un rastro. Un click, un pago, un escaneo de entrega, un ticket de soporte, una lectura de sensor.

Nadie se propone "recolectar datos". El dato es un *subproducto* de que el negocio esté funcionando. Tu trabajo, en cada rol que cubre este handbook, es tomar ese subproducto y convertirlo en algo sobre lo que la empresa pueda actuar.

flowchart LR

```
A[Usuario hace algo] → B[La app registra un evento]
B → C[El evento llega a una base de datos]
C → D["Alguien hace una pregunta\n('¿por qué bajaron los registros?')"]
D → E[Vos convertís registros crudos en una respuesta]
E → F[El negocio toma una decisión]
```

 **Ejemplo real de empresa** En **Spotify**, reproducir una canción, saltarla a los 4 segundos, o agregarla a una playlist son todos eventos. Ninguno se generó "para analytics" — es simplemente lo que hace la app cuando la usás. El trabajo del equipo de Data empieza recién *después* de que ese evento ya existe.

El dato solo no vale nada. Se vuelve valioso en el momento en que alguien lo convierte en una decisión — esa transformación, de evento crudo a decisión, es lo que enseña todo este handbook, capa por capa:

Usuarios → Aplicaciones → Base de Datos Operacional → ETL/ELT → Data Warehouse
 → SQL → Python → Power BI → Decisión de Negocio → Machine Learning → Deployment

Vas a ver este mismo pipeline de nuevo en el [Capítulo 02](#), donde cada flecha tiene su propio análisis en profundidad.

1.2 Los seis roles, y cómo se diferencian realmente

Los títulos de puesto en el mundo del dato son inconsistentes entre empresas — un "Data Analyst" en una empresa hace lo que un "Data Scientist" hace en otra. Pero el *trabajo en sí* se agrupa en seis trabajos reconocibles. Así se distinguen por lo que realmente hacen en su día a día, no por su título.

Rol	Pregunta central que responde	Herramientas principales	Output
BI Analyst	"¿Qué pasó, y cómo lo hago visible para todos?"	SQL, Power BI / Tableau	Dashboards, reportes recurrentes
Data Analyst	"¿Por qué pasó, y qué deberíamos hacer?"	SQL, Excel, Python, Power BI	Análisis ad-hoc, insights, presentaciones
Data Scientist	"¿Qué va a pasar, y qué tan seguros estamos?"	Python, estadística, ML	Modelos, experimentos, predicciones
Data Engineer	"¿Cómo movemos y almacenamos datos de forma confiable a escala?"	SQL, Python, Spark, Airflow, cloud	Pipelines, warehouses
Analytics Engineer	"¿Cómo hacemos modelos de datos limpios y confiables que todos puedan consultar?"	SQL, dbt, herramientas de warehouse	Modelos de datos, capa de transformación
ML Engineer	"¿Cómo logramos que un modelo funcione de forma confiable en producción?"	Python, APIs, Docker, cloud, MLOps	Modelos desplegados y monitoreados

📌 **Recuadro — la forma más rápida de distinguir dos roles** Preguntate: "*¿Qué entregan, y a quién?*" - Un **BI Analyst** le entrega un dashboard a un ejecutivo. - Un **Data Analyst** le entrega un insight a quien toma la decisión. - Un **Data Scientist** le entrega un modelo o una hipótesis testeada a un Data Analyst o a un equipo de producto. - Un **Data Engineer** le entrega tablas limpias y confiables a todos los de arriba. - Un **Analytics Engineer** le entrega modelos de datos *confiables y documentados* (no tablas crudas) a Analysts y Scientists. - Un **ML Engineer** le entrega un servicio corriendo y monitoreado al producto mismo.

Cómo se ve esto dentro de una empresa real

```

flowchart TB
    subgraph Ingenieria[Ingeniería]
        DE[Data Engineer\nconstruye pipelines y warehouse]
        AE[Analytics Engineer\nmodela el warehouse en\ntablas limpias y reutilizables]
    end
    subgraph Analisis[Análisis]
        BI[BI Analyst\ndashboards y reportes]
        DA[Data Analyst\ninvestigación e insight ad-hoc]
    end
    subgraph Ciencia[Ciencia]
        DS[Data Scientist\nmodelado y experimentación]
        MLE[ML Engineer\nllevar modelos a producción]
    end
    DE --> AE --> BI
    AE --> DA
    AE --> DS
    DS --> MLE
    BI --> Biz[Decisión de Negocio]
    DA --> Biz
    MLE --> Prod[Feature en Vivo del Producto]

```

 **Ejemplo real de empresa** En **Mercado Libre**, cuando un Product Manager pregunta "¿por qué sube el abandono de carrito en Brasil?", un **Data Analyst** lo responde con SQL y una presentación en días. Si la respuesta se convierte en "deberíamos construir un modelo que prediga quién está por abandonar su carrito," eso se vuelve un proyecto de **Data Scientist** — y si sale a producción como un disparador de descuento en vivo en la app, un **ML Engineer** lo pone en producción y lo mantiene funcionando.

1.3 Responsabilidades diarias, rol por rol

BI Analyst

- Mantener y extender dashboards recurrentes (revenue, retención, métricas operativas)
- Ser dueño de las definiciones de métricas para que "revenue" signifique lo mismo para todos
- Atender tickets de "el dashboard se ve mal"
- Presentar revisiones de negocio semanales/mensuales

Data Analyst

- Que lo metan en un hilo de Slack con una pregunta vaga ("¿estamos perdiendo plata con esta promo?") y convertirla en un análisis acotado
- Escribir SQL contra el warehouse, verificar los números, visualizarlos
- Presentar hallazgos con una recomendación clara, no solo un gráfico

- Trabajar directamente con stakeholders de Producto/Marketing/Finanzas

Data Scientist

- Enmarcar un problema de negocio como un problema de modelado (clasificación, regresión, ranking, etc.)
- Explorar datos, crear features, entrenar y validar modelos
- Diseñar y leer resultados de tests A/B
- Comunicar incertidumbre y trade-offs a stakeholders no técnicos

Data Engineer

- Construir y mantener pipelines de ETL/ELT
- Diseñar esquemas de warehouse para confiabilidad y performance de queries
- Ser dueño de chequeos de calidad de datos, alertas y SLAs de pipelines
- Gestionar infraestructura cloud y sus costos

Analytics Engineer

- Escribir y testear lógica de transformación (comúnmente en dbt) sobre datos crudos del warehouse
- Documentar tablas y columnas para que el analytics self-serve realmente funcione
- Ser dueño de la capa de "única fuente de verdad" entre el dato crudo y todos los que consultan
- Ser puente entre Data Engineering y Análisis

ML Engineer

- Tomar el notebook de un Data Scientist y convertirlo en un servicio confiable
- Construir pipelines de entrenamiento/inferencia, APIs, monitoreo y triggers de reentrenamiento
- Gestionar versionado de modelos y rollback
- Ser dueño de latencia, costo y uptime del ML en producción

1.4 Caminos de carrera

Ninguno de estos roles es un callejón sin salida — son un grafo, no una escalera. El punto de entrada más común para la audiencia de este handbook es **Data Analyst**, y los dos caminos dominantes desde ahí son:

```
flowchart LR
```

```
Start([Data Analyst Junior]) → DA[Data Analyst]
DA → SDA[Data Analyst Senior]
SDA → AE[Analytics Engineer]
SDA → DS[Data Scientist]
AE → DEng[Data Engineer]
```

DS → MLE[ML Engineer]
 SDA → LeadA[Analytics Manager]
 DS → LeadD[Data Science Manager]

✅ **Buena práctica** No intentes elegir tu destino final el primer día. Empezá como Data Analyst, volverte fluido en SQL y contexto de negocio, y dejá que el trabajo mismo te diga si disfrutás más el lado de "construir sistemas confiables" (→ Analytics/Data Engineering) o el lado de "modelar incertidumbre" (→ Data Science).

⚠️ **Error común** Saltar directo a tutoriales de Machine Learning sin haber consultado nunca una base de datos de producción real, sucia y sin documentar. Las empresas no contratan ML Engineers que no puedan manejar datos sucios — contratan Analysts que crecieron hacia eso. Los [Capítulos 03–05](#) asumen que pasaste por ahí antes exactamente por esta razón.

1.5 Rangos salariales (referencias orientativas)

Los salarios varían enormemente según país, tamaño de empresa y seniority — tratá esto como **orientativo**, no contractual.

Rol	LatAm (remoto, USD/año)	EE.UU. (presencial/remoto, USD/año)	UE (presencial/remoto, EUR/año)
BI Analyst	\$18k – \$40k	\$65k – \$100k	€35k – €60k
Data Analyst	\$20k – \$45k	\$70k – \$110k	€38k – €65k
Data Analyst Senior	\$35k – \$65k	\$100k – \$140k	€55k – €85k
Data Scientist	\$30k – \$60k	\$110k – \$165k	€55k – €95k
Data Engineer	\$30k – \$65k	\$115k – \$170k	€55k – €95k
Analytics Engineer	\$30k – \$60k	\$110k – \$155k	€55k – €90k
ML Engineer	\$35k – \$75k	\$130k – \$190k	€65k – €110k

📌 **Recuadro** La compensación total (equity, bonos) en grandes empresas de tecnología (Google, Amazon, Microsoft, Stripe) puede llevar el extremo superior del rango de EE.UU. entre 30–60% más alto. Los rangos de LatAm de arriba asumen **trabajo remoto para una empresa extranjera** — los salarios solo-mercado-local son típicamente 40–60% más bajos.

1.6 Matriz de habilidades requeridas

Habilidad	BI Analyst	Data Analyst	Data Scientist	Data Engineer	Analytics Engineer	ML Engineer
SQL	●●●	●●●	●●○	●●●	●●●	●●○
Excel / Planillas	●●●	●●●	●○○	●○○	●○○	○○○
Python	●○○	●●○	●●●	●●●	●●○	●●●
Estadística	●○○	●●○	●●●	●○○	●○○	●●○
Visualización de datos (Power BI/Tableau)	●●●	●●●	●○○	○○○	●○○	○○○
Modelado de Datos / Warehousing	●●○	●●○	●○○	●●●	●●●	●○○
Machine Learning	○○○	●○○	●●●	○○○	○○○	●●●
Ingeniería de Software / APIs	○○○	○○○	●○○	●●●	●●○	●●●
Comunicación de negocio	●●●	●●●	●●○	●○○	●●○	●○○
Plataformas Cloud (AWS/Azure/GCP)	●○○	●○○	●●○	●●●	●●○	●●●

●●● central al rol · ●●○ importante · ●○○ útil · ○○○ rara vez necesaria

Este handbook construye estas habilidades en el orden que realmente necesitan las empresas: **Bases de Datos** → **SQL** → **Python** → **Power BI** → **Proyectos** → **Machine Learning** → **Portfolio**, siguiendo el pipeline del §1.1.

1.7 Libros recomendados

Libro	Autor	Mejor para
<i>SQL for Data Analysis</i>	Cathy Tanimura	Fundamentos de SQL con patrones analíticos reales
<i>Storytelling with Data</i>	Cole Nussbaumer Knaflic	Convertir gráficos en decisiones sobre las que la gente actúa
<i>Python for Data Analysis</i>	Wes McKinney (creador de Pandas)	La referencia definitiva de Pandas
<i>The Data Warehouse Toolkit</i>	Ralph Kimball	Star schema, modelado dimensional — fundamental para el Cap. 03

Libro	Autor	Mejor para
<i>Practical Statistics for Data Scientists</i>	Peter Bruce, Andrew Bruce, Peter Gedeck	Estadística con mirada de practicante, no de profesor
<i>Designing Data-Intensive Applications</i>	Martin Kleppmann	Para cuando quieras entender qué construyen realmente los Data Engineers
<i>Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow</i>	Aurélien Géron	La referencia práctica estándar de ML

1.8 Canales de YouTube recomendados

Canal	Enfoque
Luke Barousse	Consejos de carrera para Data Analyst, insight real del mercado laboral
Alex The Analyst	SQL, Power BI y fundamentos de Data Analyst desde cero
StatQuest with Josh Starmer	Conceptos de estadística y ML explicados visualmente, sin relleno
Seattle Data Guy	Data Engineering y estrategia de carrera desde un practicante de la industria
Ken Jee	Caminos de carrera en Data Science y construcción de portfolio
Corey Schafer	Fundamentos de Python profundos y correctos
Guy in a Cube	Power BI directo de un ex miembro del equipo de Power BI de Microsoft

Resumen del capítulo

- El dato no tiene valor hasta que se convierte en una decisión — esa transformación es el trabajo, en cada rol de este campo.
- BI Analyst, Data Analyst, Data Scientist, Data Engineer, Analytics Engineer y ML Engineer son seis trabajos distintos que se distinguen por *qué entregan y a quién*, no solo por el título.
- La mayoría de las personas que entran a este campo deberían empezar como **Data Analyst** y ramificarse hacia Data Science o Analytics/Data Engineering según qué parte del trabajo disfruten.
- Las habilidades se acumulan en un orden específico — este handbook sigue ese orden empezando en el [Capítulo 02](#).

Siguiente: [02. Cómo Trabajan las Empresas](#) → — cómo se genera y almacena realmente el dato antes de que alguien lo pueda analizar.

02 · Cómo Trabajan las Empresas

Parte 2 · Anterior: [01. Introducción](#)

💡 **Qué vas a poder hacer después de este capítulo** Dibujar y explicar la arquitectura de datos real de una empresa moderna — desde que un usuario toca un botón hasta que un ejecutivo ve un número en un dashboard — y saber cuándo una empresa recurre a un Data Warehouse vs. un Data Lake vs. un Lakehouse, y batch vs. streaming.

2.1 Cómo se genera el dato

Toda empresa corre sobre **aplicaciones** — la app móvil, el sitio web, el sistema de punto de venta, la herramienta interna de administración. Esas aplicaciones necesitan recordar cosas *ahora mismo*: si el pago se acreditó, si el ítem sigue en stock, si el usuario está logueado. Esa memoria inmediata y transaccional vive en una **base de datos operacional** (también llamada OLTP — ver [Capítulo 03](#)).

flowchart LR

U[Usuario] → |hace click / compra / se registra| App[Aplicación]

App → |lee/escribe filas| DB[(Base de Datos Operacional\nOLTP)]

App2[Servicios backend] → |escribe eventos| DB

📖 **Ejemplo real de empresa** Cuando agregás un ítem al carrito en **Mercado Libre**, esa escritura va directo a una base de datos operacional optimizada para *velocidad y consistencia de una sola transacción* — no para responder "¿cuáles fueron nuestras 10 categorías top el mes pasado?". Esa pregunta necesita un sistema completamente distinto, que es de lo que trata el resto de este capítulo.

2.2 Base de Datos Operacional → Data Warehouse

Las bases de datos operacionales están construidas para manejar miles de transacciones pequeñas y rápidas de lectura/escritura. Son **malas** para responder grandes preguntas analíticas ("revenue total por región, por mes, en los últimos 3 años") porque ese tipo de query escanea enormes cantidades de datos — y correr eso contra la base de datos en vivo ralentizaría la app para usuarios reales.

Entonces las empresas copian el dato hacia un segundo sistema, construido específicamente para análisis: el **Data Warehouse**.

flowchart LR

DB[(Base de Datos Operacional\nOLTP)] → [ETL / ELT] DW[(Data Warehouse\nOLAP)]

DW → SQL[Análisis SQL]

DW → BI[Power BI / Tableau]

DW → DS[Data Science / ML]

El proceso que mueve y remodela el dato entre estos dos sistemas se llama **ETL** o **ELT**:

	ETL (Extract, Transform, Load)	ELT (Extract, Load, Transform)
Orden	Transforma <i>antes</i> de cargar	Transforma <i>después</i> de cargar
Dónde ocurre la transformación	Un motor de procesamiento separado	Dentro del warehouse mismo
Común hoy porque	Los warehouses ahora son lo suficientemente potentes para transformar datos a escala	Los warehouses en la nube (Snowflake, BigQuery, Redshift) son baratos para correr grandes transformaciones
Herramientas típicas	Informatica, SSIS legacy, scripts a medida	Airbyte/Fivetran (extract+load) + dbt (transform)

✓ **Buena práctica** Las empresas modernas cloud-first por defecto usan **ELT**: aterrizar el dato crudo en el warehouse primero, y después transformarlo con SQL/dbt donde es fácil de testear, versionar y volver a correr. Esta también es la capa donde viven los **Analytics Engineers** (ver [01.2](#)).

⚠ **Error común** Consultar directamente la base de datos operacional (de producción) para analytics. Así es como los analistas junior sin querer ralentizan — o tumban — la app en vivo. Analizá siempre desde el warehouse o una réplica de lectura, nunca desde la base de datos de producción primaria.

2.3 Data Warehouse vs. Data Lake vs. Lakehouse

	Data Warehouse	Data Lake	Lakehouse
Forma del dato	Estructurado (tablas, filas, columnas)	Cualquier forma — estructurado, semi-estructurado, archivos crudos, imágenes, logs	Estructurado + no estructurado, unificado
Esquema	Forzado al escribir (schema-on-write)	Forzado al leer (schema-on-read)	Ambos, según la capa

	Data Warehouse	Data Lake	Lakehouse
Mejor para	Dashboards de BI, analytics SQL	Datos de entrenamiento de ML, logs crudos, exploración de data science	Todo — una sola plataforma para ambos
Ejemplos	Snowflake, BigQuery, Redshift	S3 + Hive/Glue, Azure Data Lake	Databricks, Snowflake (Iceberg), BigQuery (unificado)

 **Ejemplo real de empresa** Una empresa como **Netflix** vuelca logs crudos de eventos de visualización en un **Data Lake** porque el volumen y la variedad son enormes y no todos los equipos lo necesitan estructurado. El equipo de BI después construye una capa curada y estructurada encima — ya sea un **Data Warehouse** clásico o, cada vez más, un **Lakehouse** que permite que tanto analistas SQL como equipos de ML trabajen sobre el mismo almacenamiento subyacente sin duplicar datos.

2.4 Batch vs. Streaming

	Procesamiento Batch	Streaming
El dato se mueve	Según un cronograma (cada hora, cada noche)	Continuamente, a medida que ocurren los eventos
Latencia	Minutos a horas	Milisegundos a segundos
Herramientas típicas	Airflow + jobs SQL/dbt, scripts de Python programados	Kafka, Kinesis, Flink, Spark Streaming
Caso de uso típico	Dashboard diario de revenue, reportes mensuales	Detección de fraude, recomendaciones en tiempo real, inventario en vivo

 **Recuadro** La mayoría del trabajo de **Data Analyst** ocurre sobre datos procesados en **batch** — un dashboard que se refresca cada mañana es completamente normal y esperado. Streaming es más una preocupación de Data/ML Engineering, algo que vas a encontrar pero rara vez vas a construir vos mismo como analista.

2.5 Plataformas cloud: AWS, Azure, Google Cloud

No necesitás dominar ningún proveedor cloud para empezar como Data Analyst, pero vas a escuchar constantemente estos nombres — sabé para qué son.

Proveedor	Warehouse	Almacenamiento (Lake)	Herramienta de BI	Notas
AWS	Redshift	S3	QuickSight (poco común)	El cloud más común en general; a menudo combinado con Power BI o Tableau
Azure	Synapse Analytics	Azure Data Lake Storage	Power BI (nativo)	Común en empresas que ya usan el stack de Microsoft
Google Cloud	BigQuery	Cloud Storage	Looker / Looker Studio	Popular en empresas tech nativas de alto crecimiento

✅ **Buena práctica** Cuando veas un aviso de trabajo que dice "SQL + BigQuery" o "SQL + Snowflake," no te asustes — el SQL que aprendés en el [Capítulo 04](#) es 90% transferible. Las diferencias son mayormente sintaxis de dialecto y cómo te conectás, no cómo pensás las queries.

2.6 El panorama completo

```

flowchart TB
    U[Usuarios] --> App[Aplicaciones]
    App --> OLTP["Base de Datos Operacional<br/>OLTP"]
    OLTP --> ELT["ELT"]
    ELT --> DW["Data Warehouse"]
    Lake["Data Lake<br/>ncrudo/no estructurado"] --> DW
    DW --> SQLA[SQL]
    SQLA --> PY[Python]
    PY --> PBI[Power BI]
    PBI --> Dec[Decisión de Negocio]
    PY --> ML[Machine Learning]
    ML --> Dep[Deployment]
    Dep --> App
  
```

Este es el diagrama que hay que tener en la cabeza para el resto del handbook — cada capítulo restante cae en un nodo de este: [Bases de Datos](#) y [SQL](#) viven en la capa del warehouse, [Python](#) y [Power BI](#) se apoyan encima, y [Machine Learning](#) cierra el loop de vuelta al producto.

Resumen del capítulo

- Las aplicaciones escriben en **bases de datos operacionales** rápidas y transaccionales — nunca analices directamente contra ellas.
- ETL/ELT** mueve y remodela ese dato hacia un **Data Warehouse** construido para análisis; las empresas modernas favorecen ELT con SQL/dbt.

- Los **Data Lakes** guardan datos crudos/no estructurados a escala; los **Lakehouses** intentan unificar ambos mundos.
- La mayoría del trabajo de analista corre sobre datos **batch**; **streaming** es más una preocupación de Data/ML Engineering.
- AWS, Azure y GCP ofrecen cada uno un warehouse, un lake y (en el caso de Azure) una herramienta de BI nativa — las habilidades de SQL que vas a construir en el Capítulo 04 se transfieren a todos ellos.

Siguiente: [03. Bases de Datos →](#) — qué hay realmente dentro de esa caja de warehouse, y cómo está estructurada para que SQL sea rápido.

03 · Bases de Datos

Parte 3 · Anterior: [02. Cómo Trabajan las Empresas](#)

💡 **Qué vas a poder hacer después de este capítulo** Leer y diseñar un esquema relacional, explicar la normalización, y diseñar un Star Schema — la habilidad que hace que todo en el [Capítulo 04 \(SQL\)](#) encaje.

3.1 Bases de datos relacionales, en una imagen

Una base de datos relacional guarda el dato en **tablas** (filas × columnas), y las tablas se relacionan entre sí a través de **claves**.

```
erDiagram
    CLIENTES ||--o{ PEDIDOS : realiza
    PEDIDOS ||--|{ ITEMS_PEDIDO : contiene
    PRODUCTOS ||--o{ ITEMS_PEDIDO : "aparece en"

    CLIENTES {
        int cliente_id PK
        string nombre
        string email
        date fecha_registro
    }
    PEDIDOS {
        int pedido_id PK
        int cliente_id FK
        date fecha_pedido
        string estado
    }
    ITEMS_PEDIDO {
        int item_id PK
        int pedido_id FK
        int producto_id FK
        int cantidad
        decimal precio_unitario
    }
    PRODUCTOS {
        int producto_id PK
        string nombre_producto
        string categoria
    }
```

```
    decimal precio
}
```

Claves Primarias y Claves Foráneas

- **Clave Primaria (PK):** la columna (o conjunto de columnas) que identifica de forma única cada fila de una tabla. `cliente_id` en `CLIENTES`.
- **Clave Foránea (FK):** una columna en una tabla que apunta a una Clave Primaria en otra, creando una relación. `cliente_id` en `PEDIDOS` apunta de vuelta a `CLIENTES`.

 **Ejemplo real de empresa** En cualquier empresa de e-commerce, `pedidos.cliente_id` es una clave foránea hacia `clientes.cliente_id`. Así es *cómo* el sistema sabe qué cliente hizo qué pedido — y es exactamente la relación sobre la que vas a hacer `JOIN` constantemente en el [Capítulo 04](#).

Índices

Un **índice** es una estructura de búsqueda que le permite a la base de datos encontrar filas sin escanear la tabla entera — la misma idea que el índice de un libro te permite saltar directo a una página en vez de leer de tapa a tapa.

 **Buena práctica** Las Claves Primarias se indexan automáticamente. Como analista, la decisión de índices que más te afecta es: las columnas que constantemente filtrás o usás para hacer join (`WHERE cliente_id = ...`, `JOIN ... ON pedido_id`) se benefician de un índice. Normalmente no vas a crear índices vos mismo, pero vas a diagnosticar queries lentas preguntándote "¿le falta uno a esto?" — cubierto en [04.7 Performance y Optimización](#).

3.2 Normalización

La **normalización** es el proceso de organizar tablas para eliminar datos duplicados e inconsistencias.

Forma	Regla	Soluciona
1FN	Cada columna contiene un solo valor (sin listas separadas por comas en una celda)	Grupos repetidos en un campo
2FN	1FN + cada columna no-clave depende de la <i>totalidad</i> de la clave primaria	Dependencia parcial en claves compuestas
3FN	2FN + ninguna columna no-clave depende de otra columna no-clave	Dependencia transitiva (ej: guardar <code>ciudad</code> y <code>país</code> cuando <code>ciudad</code> ya determina <code>país</code>)

⚠ **Error común** Guardar `nombre_cliente` en cada fila de la tabla `PEDIDOS` en vez de solo `cliente_id`. Ahora, si un cliente cambia su nombre, tenés que actualizarlo en mil lugares — y si te olvidás de uno, tu dato ahora es inconsistente. La normalización existe exactamente para prevenir esto.

🏢 **Ejemplo real de empresa** Las bases de datos operacionales (los sistemas OLTP detrás de la app) casi siempre están fuertemente normalizadas — eso mantiene las escrituras rápidas y consistentes. Pero el *warehouse* a menudo **desnormaliza** intencionalmente partes de esa estructura, porque los analistas valoran la simplicidad de las queries y la velocidad de lectura por encima de la eficiencia de escritura. Ese trade-off es exactamente para lo que sirve un Star Schema.

3.3 Star Schema vs. Snowflake Schema

Una vez que el dato llega al warehouse, típicamente se remodela alrededor de tablas de **Hechos (Fact)** y **Dimensiones (Dimension)**.

- **Tabla de hechos (Fact table):** el "qué pasó" — una fila por evento/transacción, mayormente números (medidas) y claves foráneas.
- **Tabla de dimensión (Dimension table):** el "quién/qué/dónde/cuándo" — atributos descriptivos por los que segmentás y filtrás.

```
erDiagram
    FACT_VENTAS ||--o{ DIM_FECHA : "ocurrió en"
    FACT_VENTAS ||--o{ DIM_CLIENTE : "vendido a"
    FACT_VENTAS ||--o{ DIM_PRODUCTO : "de producto"
    FACT_VENTAS ||--o{ DIM_TIENDA : "vendido en"

    FACT_VENTAS {
        int venta_id PK
        int fecha_id FK
        int cliente_id FK
        int producto_id FK
        int tienda_id FK
        decimal revenue
        int cantidad
    }

    DIM_FECHA { int fecha_id PK, date fecha_completa, int anio, int mes, string dia_semana }
    DIM_CLIENTE { int cliente_id PK, string nombre, string segmento }
    DIM_PRODUCTO { int producto_id PK, string nombre, string categoria }
    DIM_TIENDA { int tienda_id PK, string ciudad, string region }
```

Esto es un **Star Schema**: una tabla de hechos central rodeada directamente por tablas de dimensión planas. Se llama así porque el diagrama parece una estrella.

Un **Snowflake Schema** lleva esto más lejos normalizando las dimensiones mismas — ej: dividiendo DIM_PRODUCTO en DIM_PRODUCTO → DIM_CATEGORIA → DIM_DEPARTAMENTO .

	Star Schema	Snowflake Schema
Dimensiones	Desnormalizadas (planas)	Normalizadas (divididas en subtablas)
Complejidad de queries	Más simple — menos joins	Requiere más joins
Velocidad de queries	Generalmente más rápida	Puede ser más lenta por los joins extra
Almacenamiento	Levemente más redundante	Más eficiente en almacenamiento
Cuándo lo eligen las empresas	Opción por defecto para BI/reportes — favorece la simplicidad del analista	Dimensiones muy grandes, o necesidades estrictas de gobierno de datos

✅ **Buena práctica** Como Data Analyst, esperá por defecto un **Star Schema** en un warehouse bien construido — es lo que los **Analytics Engineers** optimizan, porque es lo que hace rápido y entendible tanto a Power BI como a SQL. Si te dan un Snowflake Schema, esperá más joins en tus queries del día a día.

3.4 OLTP vs. OLAP

	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Propósito	Hacer funcionar el negocio, transacción por transacción	Analizar el negocio, de forma agregada
Sistema ejemplo	La base de datos de producción de la app	El Data Warehouse
Patrón de query	Corto, simple, alta frecuencia (<code>INSERT</code> , <code>UPDATE</code> de una fila)	Largo, complejo, agregando sobre millones de filas
Esquema	Normalizado	Star/Snowflake (parcialmente desnormalizado)
Quién lo toca	La aplicación misma	Analysts, Scientists, herramientas de BI

Esta es la misma distinción del 02.2 — ahora sabés *por qué* los dos sistemas tienen formas tan distintas: OLTP está optimizado para **escrituras normalizadas y rápidas**; OLAP está optimizado para **lecturas agregadas y desnormalizadas, rápidas**.

3.5 Adelanto de SQL

No necesitás fluidez completa en SQL todavía (el [Capítulo 04](#) es el curso completo), pero así es como el esquema de arriba se convierte en una pregunta real:

```
-- "Revenue total por categoría de producto, último trimestre"
SELECT
  p.categoria,
  SUM(f.revenue) AS revenue_total
FROM fact_ventas f
JOIN dim_producto p ON f.producto_id = p.producto_id
JOIN dim_fecha d ON f.fecha_id = d.fecha_id
WHERE d.fecha_completa ≥ '2026-04-01' AND d.fecha_completa < '2026-07-01'
GROUP BY p.categoria
ORDER BY revenue_total DESC;
```

Fijate qué directo mapea esto al Star Schema: un `JOIN` por cada dimensión que necesitás, un `GROUP BY` por el nivel de detalle que querés. Ese es todo el sentido de diseñarlo así.

Resumen del capítulo

- Las bases de datos relacionales conectan tablas a través de **Claves Primarias** y **Claves Foráneas**; los **índices** hacen rápidas las búsquedas.
- La **normalización** (1FN–3FN) elimina duplicación e inconsistencia — crítica para sistemas OLTP.
- Los warehouses remodelan el dato en tablas de **Hechos** y **Dimensión**, típicamente como un **Star Schema** (más simple, más rápido) o un **Snowflake Schema** (más normalizado, más joins).
- **OLTP** hace funcionar el negocio transacción por transacción; **OLAP** lo analiza de forma agregada — formas distintas para trabajos distintos.

Siguiente: [04. SQL](#) → — el curso completo de SQL, construido directamente sobre los conceptos de esquema de este capítulo.

04 · SQL — El Curso Completo

Parte 4 · Anterior: [03. Bases de Datos](#)

 **Qué vas a poder hacer después de este capítulo** Escribir SQL analítico real de punta a punta — filtrando, agregando, uniendo, usando funciones de ventana y optimizando — contra el Star Schema del [Capítulo 03](#). Esta es la habilidad de mayor impacto en todo el handbook.

Todos los ejemplos usan el esquema `fact_ventas` / `dim_producto` / `dim_cliente` / `dim_fecha` / `dim_tienda` introducido en [03.3](#).

4.1 SELECT, WHERE, ORDER BY

```
SELECT nombre_producto, categoria, precio
FROM dim_producto
WHERE categoria = 'Electrónica'
ORDER BY precio DESC;
```

 **Caso de negocio:** Marketing pide "todos los productos de Electrónica con precio mayor a \$50, del más caro al más barato" para planear una promo.

Ejercicio: Listá todos los clientes que se registraron en 2025, ordenados por fecha de registro.

► Solución

4.2 GROUP BY, HAVING

`GROUP BY` colapsa filas en grupos; `HAVING` filtra *después* de la agregación (`WHERE` filtra *antes*).

```
SELECT categoria, SUM(revenue) AS revenue_total
FROM fact_ventas f JOIN dim_producto p ON f.producto_id = p.producto_id
GROUP BY categoria
HAVING SUM(revenue) > 100000;
```

 **Caso de negocio:** Finanzas quiere solo las categorías que generaron más de \$100k — las categorías chicas son ruido para esta revisión.

Ejercicio: Encontrá las tiendas con más de 500 pedidos en total.

► Solución

4.3 JOIN, UNION

Tipo de JOIN	Devuelve
INNER JOIN	Solo filas que coinciden en ambas tablas
LEFT JOIN	Todas las filas de la tabla izquierda, filas coincidentes de la derecha (NULL si no hay match)
RIGHT JOIN	Espejo de LEFT JOIN
FULL OUTER JOIN	Todas las filas de ambas, coincidentes donde sea posible

```
-- Clientes que todavía no hicieron ningún pedido (uso clásico de LEFT JOIN)
SELECT c.cliente_id, c.nombre
FROM dim_cliente c
LEFT JOIN fact_ventas f ON c.cliente_id = f.cliente_id
WHERE f.cliente_id IS NULL;
```

UNION apila resultados verticalmente (y elimina duplicados; UNION ALL los mantiene — y es más rápido).

 **Caso de negocio:** Growth quiere una lista de "nunca compró" para una campaña de descuento de primera compra — exactamente el patrón LEFT JOIN ... WHERE ... IS NULL de arriba.

Ejercicio: Combiná una lista de pedidos online y pedidos en tienda física en un solo resultado, manteniendo duplicados.

► Solución

4.4 CTEs y Subconsultas

Un **CTE** (Common Table Expression, **WITH**) nombra una subconsulta para que puedas construir una query en pasos legibles.

```
WITH revenue_mensual AS (
    SELECT DATE_TRUNC('month', d.fecha_completa) AS mes, SUM(f.revenue) AS revenue
    FROM fact_ventas f JOIN dim_fecha d ON f.fecha_id = d.fecha_id
    GROUP BY 1
)
SELECT mes, revenue,
       revenue - LAG(revenue) OVER (ORDER BY mes) AS cambio_mensual
FROM revenue_mensual
ORDER BY mes;
```

✓ **Buena práctica** Preferí CTEs por sobre subconsultas anidadas en profundidad — se leen de arriba hacia abajo como una narrativa, y cualquier analista de tu equipo puede debuggearlas pieza por pieza.

Ejercicio: Encontrá el cliente que más gastó por región usando un CTE.

► Solución

4.5 Vistas y Procedimientos Almacenados

- **Vista (View):** una query guardada de la que podés hacer **SELECT** como si fuera una tabla — genial para ocultar complejidad de otros analistas.
- **Procedimiento almacenado (Stored Procedure):** un bloque de SQL (y lógica) guardado y parametrizado al que llamás con **CALL** — lo usan más los engineers que los analistas, pero deberías reconocer uno cuando lo veas.

```
CREATE VIEW vw_revenue_mensual_categoria AS
SELECT DATE_TRUNC('month', d.fecha_completa) AS mes, p.categoria, SUM(f.revenue) AS revenue
FROM fact_ventas f
JOIN dim_fecha d ON f.fecha_id = d.fecha_id
JOIN dim_producto p ON f.producto_id = p.producto_id
GROUP BY 1, 2;
```

📊 **Caso de negocio:** En vez de que cada analista reescriba el mismo join de 3 tablas cada semana para el dashboard ejecutivo, un **Analytics Engineer** lo expone como una vista — esta es exactamente la idea de "única fuente de verdad" del [01.2](#).

4.6 Funciones de Ventana y Ranking

Las funciones de ventana calculan sobre un conjunto de filas *relacionadas con la fila actual*, sin colapsarlas como hace `GROUP BY`.

Función	Uso
<code>ROW_NUMBER()</code>	Número secuencial único por fila
<code>RANK()</code> / <code>DENSE_RANK()</code>	Ranking con (<code>RANK</code>) o sin (<code>DENSE_RANK</code>) huecos en empates
<code>LAG()</code> / <code>LEAD()</code>	Valor de una fila anterior/siguiente
<code>SUM()/AVG() OVER (...)</code>	Totales acumulados, promedios móviles

```
SELECT
  cliente_id, fecha_pedido, revenue,
  SUM(revenue) OVER (PARTITION BY cliente_id ORDER BY fecha_pedido) AS total_acumulado
FROM fact_ventas;
```

 **Caso de negocio:** "Mostrame el gasto acumulado de por vida de cada cliente a lo largo del tiempo" — la pregunta clásica de función de ventana de total acumulado, y una de las preguntas de entrevista de SQL más comunes.

Ejercicio: Para cada producto, rankeá sus ventas por revenue dentro de su categoría.

► Solución

4.7 Performance, Índices y Optimización

 **Errores comunes** - `SELECT *` en una tabla ancha del warehouse cuando necesitás 3 columnas — lee muchos más datos de los necesarios. - Filtrar sobre una función aplicada a una columna (`WHERE YEAR(fecha_pedido) = 2026`) — esto a menudo impide que la base de datos use un índice. Preferí `WHERE fecha_pedido ≥ '2026-01-01' AND fecha_pedido < '2027-01-01'`. - Hacer join sobre columnas sin índice en tablas grandes — consultá con tu Data Engineer si un camino de join muy usado está lento.

 **Buenas prácticas** - Filtrá lo antes posible (`WHERE` antes de unir más tablas de las necesarias). - Lee el **plan de ejecución** (`EXPLAIN` / `EXPLAIN ANALYZE`) cuando una query esté inesperadamente lenta — te dice si está escaneando una tabla completa o usando un índice. - Agregá al nivel más

grueso que realmente necesite la pregunta — no traigas datos fila por fila a Python solo para sumarlos ahí.

Resumen del capítulo

- `SELECT/WHERE/ORDER BY` → `GROUP BY/HAVING` → `JOIN` son el 80% diario del SQL de un analista.
- Los CTEs hacen legible la lógica de varios pasos; las Vistas exponen esa lógica como bloques reutilizables y confiables.
- Las funciones de ventana resuelven problemas de "comparar esta fila con otras filas" sin colapsar tu conjunto de resultados — rankings, totales acumulados, cambio período a período.
- La performance de queries se trata mayormente de filtrar temprano, evitar funciones sobre columnas filtradas, y leer el plan de ejecución cuando algo está lento.

Siguiente: [05. Python](#) → — donde el output de SQL se convierte en limpieza, EDA e ingeniería de features.

05 · Python

Parte 5 · Anterior: [04. SQL](#)

💡 **Qué vas a poder hacer después de este capítulo** Tomar el resultado de una query SQL y hacer todo lo que SQL hace mal: limpieza profunda, análisis exploratorio, ingeniería de features y visualización — organizado como un proyecto profesional, no como un notebook de borrador.

5.1 Dónde encaja Python (y dónde no)

🚩 **Recuadro** No traigas millones de filas a Python para hacer lo que SQL ya hace bien (agregación, filtrado, joins). Usá SQL para que el warehouse haga el trabajo pesado, y después traé el *resultado* a Python para lo que SQL no hace bien: modelado estadístico, lógica de limpieza compleja y visualización rica.

```
flowchart LR
    DW["DW[(Data Warehouse)]"] -->|"query SQL, agregada"| DF["DF[(DataFrame de Pandas)"]
    DF --> Clean["Clean[(Limpieza)"]
    Clean --> EDA["EDA[(EDA)"]
    EDA --> FE["FE[(Ingeniería de Features)"]
    FE --> Viz["Viz[(Visualización / Estadística)"]
    FE --> ML["ML[(Machine Learning)\nCapítulo 08]"]
```

5.2 Pandas y NumPy

```
import pandas as pd
import numpy as np

df = pd.read_sql("SELECT * FROM vw_revenue_mensual_categoria", conn)
df.head()
df.info()
df.describe()
```

NumPy es la base de Pandas — lo vas a usar directamente para matemática vectorizada y manejo de NaN :

```
df['revenue_log'] = np.log1p(df['revenue'])
```

 **Ejemplo real de empresa** Los analistas en empresas como **Rappi** o **Amazon** rara vez cargan tablas crudas de millones de filas en Pandas — agregan en SQL primero (ver [04.7](#)), y después traen unas pocas miles de filas resumidas a Python para el análisis que SQL no puede expresar.

5.3 Limpieza

```
df = df.drop_duplicates()
df['email'] = df['email'].str.strip().str.lower()
df['fecha_registro'] = pd.to_datetime(df['fecha_registro'], errors='coerce')
df['revenue'] = df['revenue'].fillna(0)
```

 **Error común** Descartar filas con datos faltantes con `dropna()` sin chequear *cuánto* dato estás perdiendo o *por qué*. Siempre chequea `df.isna().sum()` primero y entendé si la ausencia de datos es aleatoria o significativa (ej: "revenue es nulo" podría significar "el pedido se canceló," no "error de datos").

5.4 Análisis Exploratorio de Datos (EDA)

```
df['categoria'].value_counts()
df.groupby('categoria')['revenue'].agg(['sum', 'mean', 'count'])
df.corr(numeric_only=True)
```

El EDA responde: ¿cómo es realmente este dato, hay outliers, hay sorpresas, algo contradice lo que asumen los stakeholders?

5.5 Ingeniería de Features

```
df['mes_pedido'] = df['fecha_pedido'].dt.to_period('M')
df['es_finde'] = df['fecha_pedido'].dt.dayofweek >= 5
df['revenue_por_unidad'] = df['revenue'] / df['cantidad']
```

La ingeniería de features es donde el conocimiento del negocio se convierte en una columna que un modelo o un gráfico puede usar directamente — el puente hacia el [Capítulo 08 \(Machine Learning\)](#).

5.6 Visualización y Estadística

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.lineplot(data=df, x='mes_pedido', y='revenue', hue='categoria')
plt.title('Revenue Mensual por Categoría')
plt.show()
```

Acompaña cada gráfico con la estadística que lo respalda — un promedio sin una noción de dispersión (desvío estándar, IQR) puede confundir a un stakeholder.

✅ **Buena práctica** Preguntate siempre "¿comparado con qué?" Un número solo ("el revenue fue \$2M el mes pasado") no significa nada sin una comparación — mes contra mes, año contra año, o contra un objetivo.

5.7 Organización de proyectos, logging, entornos virtuales

```
proyecto/
├── data/
├── notebooks/
├── src/
│   ├── clean.py
│   ├── features.py
│   └── analysis.py
├── requirements.txt
├── .gitignore
└── README.md
```

```
python -m venv .venv
source .venv/bin/activate      # o .venv\Scripts\activate en Windows
pip install pandas numpy matplotlib seaborn
pip freeze > requirements.txt
```

```
import logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
logger.info(f"Se cargaron {len(df)} filas después de limpiar")
```

✅ **Buena práctica** Un **entorno virtual** por proyecto + un `requirements.txt` versionado es lo que separa "un script que corrió en mi máquina una vez" de un proyecto que un compañero

realmente puede reproducir. Este es exactamente el hábito en el que te vas a apoyar para los [10 proyectos de portfolio](#).

Resumen del capítulo

- Dejá que SQL agregue; dejá que Python limpie, explore, cree features y visualice.
- Siempre cuantificá los datos faltantes antes de descartarlos, y siempre comparás números contra algo.
- Estructurá cada proyecto con `venv` + `requirements.txt` + logging desde el día uno — es un hábito de cinco minutos que hace tu trabajo reproducible.

Siguiente: [06. Power BI →](#) — convirtiendo este análisis en un dashboard que los stakeholders realmente usen.

06 · Power BI

Parte 6 · Anterior: [05. Python](#)

💡 **Qué vas a poder hacer después de este capítulo** Construir un dashboard real desde cero: cargar y modelar datos, escribir medidas DAX, y publicar algo en lo que un Product Manager o un ejecutivo realmente confiaría.

6.1 Power Query

Power Query es la capa de carga y transformación de datos de Power BI (Obtener Datos → Transformar). Pensalo como una interfaz gráfica para muchos de los pasos de limpieza que hiciste en el [Capítulo 05](#), excepto que acá se compilan en un pipeline reutilizable y actualizable.

✅ **Buena práctica** Hacé la transformación pesada río arriba (SQL/dbt en el warehouse) y dejá Power Query para transformaciones livianas — renombrar, convertir tipos, filtros simples. Un modelo que se apoya en el warehouse para la lógica es más fácil de debuggear y mucho más rápido de actualizar.

6.2 Modelado de Datos y Relaciones

Power BI quiere que tu dato esté modelado de la misma forma que aprendiste en el [Capítulo 03](#): un **Star Schema** con una tabla de hechos y varias tablas de dimensión, conectadas por relaciones uno-a-muchos.

```
erDiagram
    FACT_VENTAS }o--|| DIM_FECHA : ""
    FACT_VENTAS }o--|| DIM_PRODUCTO : ""
    FACT_VENTAS }o--|| DIM_CLIENTE : ""
```

⚠️ **Error común** Importar tablas anchas, planas y pre-unidas desde una planilla en vez de un star schema apropiado. Funciona para un dataset diminuto, y después se rompe rápido — filas de dimensión duplicadas, agregaciones incorrectas, y un DAX innecesariamente complicado. Modelalo bien desde el principio.

6.3 DAX, Medidas, KPIs

DAX (Data Analysis Expressions) es el lenguaje de fórmulas de Power BI. El hábito más importante: escribir **Medidas**, no **Columnas Calculadas**, para todo lo que agregues.

```
Revenue Total = SUM(fact_ventas[revenue])
```

```
Revenue % MoM =
```

```
VAR MesActual = [Revenue Total]
```

```
VAR MesAnterior =
```

```
    CALCULATE([Revenue Total], DATEADD(dim_fecha[fecha_completa], -1, MONTH))
```

```
RETURN
```

```
    DIVIDE(MesActual - MesAnterior, MesAnterior)
```

	Columna Calculada	Medida
Se calcula	Fila por fila, guardada en el modelo	Al vuelo, según el contexto de filtro
Usar para	Atributos estáticos por fila (ej: Nombre Completo)	KPIs, agregaciones, cualquier cosa que cambie con los filtros
Costo	Aumenta el tamaño del modelo	Se calcula en tiempo de consulta — sin costo de almacenamiento

 **Caso de negocio:** Un ejecutivo filtra el dashboard a "Región = LatAm, Q2." Una **Medida** bien escrita para `Revenue Total` se recalcula automáticamente para ese contexto de filtro — una columna calculada no lo haría.

6.4 Dashboards, Bookmarks, Drill Through

- **Diseño de dashboard/reporte:** lidera con el KPI, respaldá con la tendencia, permití el drill-down — no enterrés la respuesta debajo de cinco gráficos de contexto.
- **Bookmarks:** guardan un estado específico de filtro/vista, se usan para construir "botones" guiados que cambian vistas del reporte.
- **Drill Through:** dejá que un usuario haga click derecho en un punto de dato (ej: una región) y salte a una página de detalle filtrada solo a esa región.

✓ **Buena práctica** Diseña para la "lectura de 5 segundos": alguien que le echa un vistazo a tu dashboard en medio de una reunión debería obtener el número principal y su dirección (arriba/abajo vs. objetivo) sin tener que buscar.

6.5 Deployment, Publicación, Buenas Prácticas

flowchart LR

```
PBIX[Power BI Desktop\narchivo .pbix] -->|Publicar| Service[Power BI Service]
Service -->|Actualización programada| DW[(Data Warehouse)]
Service -->|Compartir / Embeber| Stakeholders[Stakeholders]
```

✅ **Checklist de buenas prácticas** - Configuraré la **actualización programada** para que el dashboard nunca muestre datos desactualizados. - Usé **Seguridad a Nivel de Fila (RLS)** si distintas regiones/equipos solo deberían ver sus propios datos. - Documentaré tus medidas (Power BI soporta descripciones) para que el próximo analista no tenga que hacer ingeniería inversa de tu DAX. - Versionaré tus archivos `.pbix` y mantendré un changelog para rediseños importantes del dashboard, la misma disciplina que el [CHANGELOG.md](#) de este handbook.

Resumen del capítulo

- Power Query da forma liviana al dato; la transformación pesada pertenece río arriba, en SQL.
- Modelaré en Star Schema — es la misma disciplina del Capítulo 03, ahora dentro de Power BI.
- Escribí **Medidas**, no Columnas Calculadas, para los KPIs — respetan el contexto de filtro.
- Publicaré con actualización programada, RLS donde haga falta, y documentación, para que el dashboard sobreviva sin que vos estés en la sala.

Siguiente: [07. Proyectos Profesionales](#) → — combinando SQL, Python y Power BI en un proyecto completo y presentable.

07 · Proyectos Profesionales

Parte 7 · Anterior: [06. Power BI](#)

 **Qué vas a poder hacer después de este capítulo** Definir el alcance, estructurar y entregar un proyecto de la forma en que realmente pasa dentro de una empresa — no un notebook de Kaggle, sino un problema de negocio llevado desde la pregunta hasta el resumen ejecutivo.

7.1 La metodología

Cada proyecto en [../proyectos/](#) sigue el mismo arco de siete pasos — que no es casualidad, es también cómo se define el alcance del trabajo real de un analista en una empresa:

flowchart LR

```
A[1. Problema de Negocio] → B[2. Datos y Esquema]
B → C[3. Preguntas a Responder]
C → D[4. SQL]
D → E[5. Python]
E → F[6. Power BI]
F → G[7. Resumen Ejecutivo]
```

1. **Problema de Negocio** — un párrafo, escrito como lo preguntaría realmente un stakeholder ("¿Por qué está subiendo el churn en el Noreste?"), no como una descripción de dataset.
2. **Diagrama de Base de Datos** — el esquema con el que estás trabajando, dibujado como en el [Capítulo 03](#).
3. **Preguntas a Responder** — 4–6 preguntas concretas en las que se descompone el problema de negocio.
4. **SQL** — extracción y agregación según el [Capítulo 04](#).
5. **Python** — limpieza, EDA e ingeniería de features según el [Capítulo 05](#).
6. **Power BI** — el dashboard que los stakeholders realmente van a abrir, según el [Capítulo 06](#).
7. **Resumen Ejecutivo** — una página: qué encontraste, qué recomendás, qué necesitarías para ir más allá.

 **Buena práctica** Escribí el Resumen Ejecutivo **primero**, como hipótesis, antes de tocar el dato. Te obliga a plantear cómo sería una buena respuesta — y después el análisis o la confirma, o la mata, o la complica. Eso da una narrativa de proyecto mucho más sólida que "exploré el dato y encontré algunas cosas".

⚠ **Error común** Liderar un proyecto con las herramientas ("Usé Python y Power BI para analizar datos de retail") en vez del problema de negocio. Los reclutadores y hiring managers hojean docenas de estos — los que se leen como un caso de negocio, no como una vidriera de herramientas, se destacan.

7.2 Estructura de carpetas del proyecto

Cada proyecto en `../proyectos/` usa esta estructura:

```
proyectos/<NN_Nombre_Proyecto>/
├─ README.md           # problema de negocio, objetivos, arquitectura, workflow, conclusiones
├─ data/
├─ sql/
├─ python/
├─ powerbi/
├─ diagramas/
└─ presentacion/
```

7.3 Los 10 proyectos

#	Proyecto	Dificultad	Habilidades principales
01	Analítica de Ventas Retail	Principiante	SQL, Power BI
02	Churn de Clientes E-commerce	Intermedio	SQL, Python
03	BI de Adventure Works	Intermedio	SQL, Power BI
04	Operaciones Northwind	Principiante	SQL
05	Detección de Fraude con Tarjeta de Crédito	Avanzado	Python, ML
06	Analítica de Rotación de Personal (HR)	Intermedio	SQL, Python, Power BI
07	Performance de Campañas de Marketing	Intermedio	SQL, Power BI
08	Forecasting Financiero	Avanzado	Python, Series de Tiempo
09	Motor de Recomendación de Productos	Avanzado	Python, ML
10	Capstone: Pipeline End-to-End	Avanzado	SQL, Python, Power BI, ML

Resumen del capítulo

- Un proyecto real empieza con un problema de negocio planteado como lo preguntaría un stakeholder, no como una descripción de dataset.
- Estructura: Problema de Negocio → Esquema → Preguntas → SQL → Python → Power BI → Resumen Ejecutivo.
- Escribí el Resumen Ejecutivo como hipótesis primero — afila todo el análisis.
- Los 10 proyectos en [../proyectos/](#) aplican esta metodología de punta a punta, aumentando en dificultad.

Siguiente: [08. Machine Learning](#) → — para los proyectos (05, 08, 09, 10) que van más allá de SQL/BI hacia el modelado predictivo.

08 · Machine Learning

Parte 8 · Anterior: [07. Proyectos Profesionales](#)

 **Qué vas a poder hacer después de este capítulo** Enmarcar un problema de negocio como un problema de modelado, construir un pipeline de evaluación apropiado, y entender qué hace falta para llevar un modelo a producción — el puente de Data Analyst → Data Scientist del [01.4](#).

8.1 Los tipos de problema

Tipo	Predice	Ejemplo
Regresión	Un número continuo	Pronosticar el revenue del próximo mes
Clasificación	Una categoría	¿Este cliente va a hacer churn? (sí/no)
Clustering	Estructura sin etiquetas	Segmentar clientes por comportamiento, sin etiquetas dadas
Recomendación	Relevancia rankeada	"Los clientes que compraron X también compraron Y"
Series de Tiempo	Valores futuros en el tiempo	Pronóstico de demanda diaria
NLP	Estructura/significado de texto	Clasificar tickets de soporte por tema, sentimiento de reseñas

 **Ejemplo real de empresa** Un sistema de **Detección de Fraude con Tarjeta de Crédito** (ver [Proyecto 05](#)) es un problema de clasificación con desbalance de clases extremo — el fraude es tal vez el 0.1% de las transacciones. Enmarcar esto correctamente (y elegir la métrica correcta) importa más que qué algoritmo elijas.

8.2 Ingeniería de Features (repaso + mirada de modelado)

Todo del [05.5](#) aplica — ahora con un objetivo de modelado: ¿esta feature realmente ayuda al modelo a separar clases / predecir el número, o es ruido sobre el que el modelo va a hacer overfitting?

```
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
```

```
preprocess = ColumnTransformer([
    ('num', StandardScaler(), ['revenue_por_unidad', 'cantidad_pedidos']),
    ('cat', OneHotEncoder(handle_unknown='ignore'), ['categoria', 'region']),
])
```

8.3 Pipelines y Validación Cruzada

```
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

pipe = Pipeline([
    ('preprocess', preprocess),
    ('model', RandomForestClassifier(random_state=42)),
])

scores = cross_val_score(pipe, X_train, y_train, cv=5, scoring='f1')
pipe.fit(X_train, y_train)
```

✅ **Buena práctica** Siempre envolví el preprocesamiento + modelo en un único Pipeline, y siempre hacía validación cruzada. Un modelo que se ve genial en un split de train/test y terrible en otro no está listo — esta es la forma #1 en que los proyectos junior sobreestiman sus resultados.

8.4 Evaluación de Modelos

Métrica	Usar cuando
RMSE / MAE	Regresión — magnitud promedio del error
Accuracy	Clasificación, solo cuando las clases están balanceadas
Precision / Recall / F1	Clasificación con desbalance (fraude, churn)
ROC-AUC	Qué tan bien separa el modelo las clases a lo largo de distintos umbrales

⚠️ **Error común** Reportar 99% de accuracy en un modelo de fraude donde el fraude es el 0.1% del dato — un modelo que predice "no es fraude" para todo obtiene 99.9% de accuracy siendo completamente inútil. Exactamente por esto existen **Precision/Recall**.

8.5 Optimización de Hiperparámetros

```
from sklearn.model_selection import GridSearchCV

param_grid = {'model__n_estimators': [100, 300], 'model__max_depth': [5, 10, None]}
grid = GridSearchCV(pipe, param_grid, cv=5, scoring='f1')
grid.fit(X_train, y_train)
print(grid.best_params_)
```

8.6 Deployment (el traspaso al ML Engineer)

```
flowchart LR
    NB[Notebook: modelo entrenado] --> Serialize[Serializar\n.pkl / MLflow]
    Serialize --> API[Envolver en API\nFastAPI / Flask]
    API --> Prod[Servicio en producción]
    Prod --> Monitor[Monitoreo: drift, latencia, accuracy]
    Monitor -->|dispara| Retrain[Pipeline de reentrenamiento]
```

 **Ejemplo real de empresa** El modelo de churn de un Data Scientist no genera valor de negocio sentado en un notebook. Un **ML Engineer** lo envuelve detrás de una API, el producto lo llama cuando decide quién recibe una oferta de retención, y el modelo se monitorea por **drift** — cuando el dato en vivo empieza a parecerse menos al dato de entrenamiento, el accuracy se degrada silenciosamente. Este es el límite de roles del [01.2](#).

Resumen del capítulo

- Enmarcá correctamente el problema de negocio como regresión / clasificación / clustering / reclsys / series de tiempo / NLP antes de elegir un algoritmo.
- Usá siempre un `Pipeline` + validación cruzada — nunca confíes en un solo split de train/test.
- Hací coincidir la métrica con el problema: la clasificación desbalanceada necesita Precision/Recall/F1, no accuracy.
- El deployment es una disciplina distinta (ML Engineering) — entrenar un buen modelo es necesario pero no suficiente.

Siguiente: [09. Portfolio y Carrera](#) → — convirtiendo todo lo que construiste en un trabajo.

09 · Portfolio y Carrera

Parte 9 · Anterior: [08. Machine Learning](#)

💡 **Qué vas a poder hacer después de este capítulo** Empaquetar todo lo de los Capítulos 1–8 en un portfolio y una búsqueda laboral que realmente convierta — higiene de Git, un CV que se lee como impacto en vez de tareas, preparación de entrevistas, y un camino hacia el freelancing si querés uno.

9.1 Git y GitHub

```
git init
git add .
git commit -m "Agregar análisis de churn de clientes: extracción SQL + EDA en Python"
git remote add origin <tu-url-de-repo>
git push -u origin main
```

✅ **Buena práctica** Comiteá en bloques significativos con mensajes que describan *por qué*, no "update file" — un hiring manager que hojea tu historial de commits está evaluando silenciosamente cómo vas a trabajar en su equipo. Esta es la misma disciplina que modela el propio [CHANGELOG.md](#) de este handbook.

9.2 README y Estructura de Proyecto

Cada proyecto ([Capítulo 07](#)) necesita un README que un reclutador pueda leer en 60 segundos y entender: **Cuál era el problema de negocio, qué hiciste, qué encontraste**. Empezá con una captura del dashboard o un gráfico clave — no con una pared de texto.

⚠️ **Error común** Un perfil de GitHub con 15 repos de tutoriales seguidos al pie de la letra ("Notebook Titanic #1") y cero proyectos terminados y narrados. Tres proyectos completos con un planteo real de negocio (según el [Capítulo 07](#)) le ganan a quince sin terminar, siempre.

9.3 LinkedIn y CV

Fórmula para bullets del CV: [Acción] + [qué analizaste/construiste] + [impacto de negocio cuantificado]

Débil	Fuerte
"Usé SQL y Python para analizar datos de ventas"	"Identifiqué una fuga de revenue de \$180k/año en un bug de acumulación de descuentos usando análisis de cohortes en SQL; recomendación adoptada por Finanzas"
"Construí dashboards en Power BI"	"Construí un dashboard ejecutivo en Power BI adoptado por 3 equipos regionales, reemplazando 6 horas/semana de reportes manuales"

✓ **Buena práctica** Cada bullet debería sobrevivir a la pregunta "¿y qué?". Si no termina en un resultado que le importe a alguien de negocio, reescribilo.

9.4 Preguntas de Entrevista

Ronda de SQL (ver también [Capítulo 04](#)): - Escribí una query para encontrar el segundo salario más alto por departamento. - Explicá la diferencia entre `WHERE` y `HAVING`. - Dadas dos tablas, escribí una query para encontrar clientes sin pedidos.

Ronda de Python (ver también [Capítulo 05](#)): - Dado un DataFrame desordenado, explicá en voz alta tu enfoque de limpieza. - Explicá `.groupby()` vs. una tabla dinámica — ¿cuándo usarías cada uno?

Ronda de caso de negocio: - "Los usuarios activos semanales bajaron 10% la semana pasada — contame cómo lo investigarías." - Practicá estructurar tu respuesta: aclarar la definición de la métrica → chequear problemas de datos → segmentar (por plataforma, geografía, cohorte) → formular y testear una hipótesis → recomendar próximos pasos.

Ronda de ML (ver también [Capítulo 08](#)): - Precision vs. Recall — ¿cuándo optimizás por uno sobre el otro? - ¿Cómo sabés si un modelo está haciendo overfitting?

9.5 Freelancing (Fiverr, Upwork)

El trabajo freelance en datos es una forma legítima de construir portfolio e ingresos mientras buscás trabajo:

- Empezá con proyectos chicos y bien acotados: "limpiame y analizame este dataset," "construime un dashboard de 3 páginas en Power BI."
- Cobrá por el entregable, no por hora, una vez que tengas 2-3 reseñas.
- Cada proyecto freelance es un proyecto de portfolio — aplicale la estructura del [Capítulo 07](#) también.

⚠ **Error común** Cobrar de menos para ganar el primer trabajo, y quedarte cobrando de menos para siempre. Baja el precio solo para tus primeras 2-3 reseñas, y después subí tus tarifas — las reseñas son el activo que realmente estás comprando en esa etapa.

9.6 Cronogramas de Aprendizaje

Track	Ritmo	Distribución aproximada por capítulo
6 meses (intensivo, ~25-30 hs/sem)	Cap.01–03: 2 sem · SQL: 4 sem · Python: 5 sem · Power BI: 3 sem · Proyectos: 6 sem · ML: 4 sem · Portfolio/búsqueda laboral: continuo desde el mes 4	
12 meses (balanceado, ~12-15 hs/sem)	Mismo orden, aproximadamente el doble de tiempo por fase, con 2 proyectos corriendo en paralelo con cada nueva habilidad	
18 meses (part-time, ~6-8 hs/sem)	Mismo orden, un tema a la vez, mínimo 1 proyecto entregado por trimestre	

9.7 Consejos de un Ingeniero Senior

📖 **Desde las trincheras** - "Los mejores analistas que contraté no eran los mejores en SQL — eran los mejores en preguntar '¿es esta la pregunta correcta?' antes de escribir cualquier query." - "En tus primeros seis meses en el trabajo, sobre-comunicá los supuestos. Un número incorrecto que nadie cuestionó es peor que un número lento en el que todos confían." - "No esperes permiso para tener curiosidad sobre una métrica que se ve rara. Ese instinto es el trabajo en sí."

Resumen del capítulo

- La higiene de Git/GitHub y un README que lidera con impacto de negocio son parte del entregable, no un detalle secundario.
- Los bullets del CV deberían sobrevivir a "¿y qué?" — cuantificá el impacto de negocio, no solo las herramientas usadas.
- Preparate para cuatro frentes de entrevista: SQL, Python, caso de negocio, y (si aplica) ML.
- El freelancing es un camino paralelo legítimo — tratá cada trabajo como un proyecto de portfolio.

- Elegí un cronograma (6/12/18 meses) que coincida con tus horas reales disponibles, no con tu ambición.

Esto completa el handbook central. Volvé a la Tabla de Contenidos · Continúa con los [10 proyectos de portfolio](#) para aplicar todo.